POSIX Threads & Concurrency

A practical reference manual with tested C implementations

What this guide covers: thread creation and lifecycle, shared memory, race conditions, mutexes, condition variables, bounded producer-consumer queues, deadlock prevention, graceful shutdown, thread cleanup, and debugging.

Audience: engineering learner transitioning from STM32 / FreeRTOS concepts to Linux userspace systems programming.

How to read: understand the problem and invariant first; inspect the short pattern; then read and run the complete example in the Appendix.

Five rules to carry into every design

1. Define who owns shared state. **2.** Protect every conflicting access. **3.** Wait for a predicate, not a signal. **4.** Define lock ordering. **5.** Plan shutdown before starting threads.

Edition: October 2026 | Linux userspace / POSIX pthreads

Code was compiled with GCC and example programs were run during preparation. Scheduling-dependent print order is deliberately not specified.

Reading map

Part	Core question	Complete example
01 Threads	Who runs the function? Who owns its arguments?	01_threads.c
02 Races / locks	Which data must never be accessed concurrently without coordination?	02_race_bad.c; 03_mutex_counter.c
03 Condition variables	How does a thread sleep without blocking the worker that can change state?	04_condition_variable.c
04 Lock ordering	How can lock acquisition itself stop all progress?	05_lock_order.c
05 Bounded queues	How do producers and consumers exchange data safely?	06_bounded_queue.c
06 Shutdown	How do we stop accepting jobs and drain existing work?	06_bounded_queue.c
07 Optional atomics	When is a mutex unnecessary for a single counter?	07_atomic_counter.c

Build and run: one command for every example

```
gcc -std=c11 -O0 -g -Wall -Wextra -Wpedantic -pthread 04_condition_variable.c -o demo
./demo
```

Meaning of the flags: `-std=c11` selects C11; `-O0 -g` helps debugging; `-Wall -Wextra -Wpedantic` requests additional diagnostics; `-pthread` enables the platform's pthread compilation/link settings.

Scope: these are userspace APIs

This manual teaches **POSIX pthreads in a Linux process**. The Linux kernel has different synchronization APIs. The concepts transfer, but you cannot call `pthread_mutex_lock()` inside a kernel driver.

> **The key shift from FreeRTOS**
> A FreeRTOS queue already packages synchronization and blocking. In this module, we deliberately build some of that machinery ourselves: protected shared state + a ring buffer + condition variables + shutdown rules.

01. What is a thread?

A process has an address space and operating-system resources. Threads of one process share that address space (including globals and heap objects), but each thread has its own stack, register state, and execution flow. The scheduler may interleave statements from different threads; do not assume a particular print order. [1]

The minimum lifecycle

```
pthread_t t;
WorkerArgs args = { .id = 7, .jobs = 3 };
int rc = pthread_create(&t, NULL, worker, &args);
/* After checking rc: do other work, then... */
pthread_join(t, NULL);
```

`worker` has the function signature `void *worker(void *arg)`. Its argument is a pointer, not an independent copy. The pointed-to object must remain alive until the worker has finished using it.

Important: do not pass the address of a loop-local variable that is overwritten every iteration as all workers' IDs. Allocate one stable argument object per worker (as in `01_threads.c`).

C syntax translator

Syntax	Meaning
`&x`	Address of x; pointer to x
`*p`	Object pointed to by p (dereference)
`p->field`	Access a struct field through a pointer
`void *`	Generic object pointer used as pthread argument / return type
`return NULL;`	Thread function finishes without returning a result object

02. Data races and mutexes

Consider two workers executing `counter++`. That expression is a read-modify-write operation, not a guaranteed indivisible operation. When accesses conflict without a happens-before relationship, the program has a C data race and undefined behavior. The observed counter might even look correct, which does **not** validate the code.

Wrong: shared counter with no synchronization

```
static int counter = 0;
/* Both threads run: */
for (int i = 0; i < 100000; ++i)
    counter++;  /* DATA RACE - intentionally bad */
```

Correct: one mutex guards all accesses

```
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&mutex);
counter++;              /* Critical section */
pthread_mutex_unlock(&mutex);
```

A **critical section** is the region that must not overlap with another conflicting access. The producer and consumer must follow the same locking contract for each shared object; locking only the writer

does not protect an unsynchronized reader.

Do not confuse: `volatile` is not a mutual-exclusion mechanism; `sleep()` does not make a race safe; a mutex must be acquired on every path that needs the protected invariant.

What a mutex does not guarantee

It does not promise which waiting thread runs next, guarantee fairness, or automatically make two distinct mutex-protected objects consistent with each other. Keep critical sections small; avoid slow device I/O while holding a lock unless the design requires it.

03. Condition variables: waiting correctly

A mutex answers **who may touch this state right now?** A condition variable helps answer **how can I sleep until the state might have changed?** The condition variable does not contain the condition itself: the shared predicate does (e.g., `count > 0`). [3,4]

The essential pattern

```
pthread_mutex_lock(&mutex);
while (count == 0) {
    pthread_cond_wait(&not_empty, &mutex);
}
/* Here: mutex held and count > 0. */
/* Consume an item. */
pthread_mutex_unlock(&mutex);
```

`pthread_cond_wait()` **atomically releases the mutex and begins waiting**. Before it returns, it reacquires that mutex. Use `while`, not `if`, because the predicate must be checked again after any wakeup. [3]

Consumer	Producer
`lock mutex`	
`count == 0`	
`cond_wait: unlock + sleep`	`lock mutex`
`(blocked, mutex FREE)`	`enqueue; count++`
`wake; reacquire mutex`	`signal(not_empty); unlock`
`recheck count; pop`	

Signal vs broadcast

API	Who wakes?	Typical usage
`pthread_cond_signal(&cv)`	At least one current waiter, if present	One new queue item may let a consumer proceed
`pthread_cond_broadcast(&cv)`	All current waiters	Shutdown: every waiting consumer must recheck closed

Signaling **does not transfer ownership of the mutex**, and signals are not stored like queue items. Update the protected predicate before notification. In these examples, notification happens while holding the mutex to make reasoning straightforward. [4]

04. A circular bounded queue

A fixed-size array can store FIFO jobs without shifting any array elements. We track head (next read), `tail` (next write), and `count` (number of pending jobs). Indices wrap using modulo `%`.

Physical storage (CAPACITY = 3)



Indices move; values are not shifted around the array.

```
/* Enqueue, only when count < CAPACITY and lock is held. */
queue[tail] = job;
tail = (tail + 1) % CAPACITY;
count++;

/* Dequeue, only when count > 0 and lock is held. */
job = queue[head];
head = (head + 1) % CAPACITY;
count--;
```

Invariants that must always be true

- `0 <= count <= CAPACITY`.
- Only a producer changes `tail` or adds an item; only a consumer advances head or removes one, all under the queue mutex.
- If `count == CAPACITY`, producers wait for `not_full`. If `count == 0`, consumers wait for `not_empty`, unless the queue is closed.
- `head == tail` can mean either empty or full: `count` disambiguates.

05. Bounded producer-consumer: both directions

Our full implementation is `06_bounded_queue.c`: two producers, three consumers, capacity three. It has an explicit API for pushing, popping and closing. The code is in the Appendix and was compiled and run.

Producer must check capacity, then notify the consumer

```
pthread_mutex_lock(&q->mutex);
while (q->count == CAPACITY && !q->closed)
    pthread_cond_wait(&q->not_full, &q->mutex);
if (q->closed) { /* reject new job */ }
/* insert, advance tail, count++ */
pthread_cond_signal(&q->not_empty);
pthread_mutex_unlock(&q->mutex);
```

Consumer must check availability, then notify a producer

```
pthread_mutex_lock(&q->mutex);
while (q->count == 0 && !q->closed)
    pthread_cond_wait(&q->not_empty, &q->mutex);
if (q->count == 0 && q->closed) { /* no future jobs: exit */ }
/* remove, advance head, count-- */
pthread_cond_signal(&q->not_full);
pthread_mutex_unlock(&q->mutex);
```

Why two condition variables? Producers need to know that capacity became available, while consumers need to know that a job became available. Different predicates, different reasons to wake up.

06. Deadlocks and lock ordering

A deadlock can arise when thread 1 holds A while waiting for B, and thread 2 holds B while waiting for A. Neither thread can reach its unlock statements. Different execution orders may hide the bug during testing.

Unsafe acquisition order

```
/* Thread 1 */            /* Thread 2 */
lock(A);                   lock(B);
lock(B);                   lock(A);
/* work */                 /* work */
```

Corrected acquisition order

```
/* Every thread follows the SAME ordering. */
pthread_mutex_lock(&mutex_a);
pthread_mutex_lock(&mutex_b);
/* critical section */
pthread_mutex_unlock(&mutex_b);
pthread_mutex_unlock(&mutex_a);
```

The crucial prevention rule is the **consistent acquisition order**. Unlocking in reverse is a useful convention. The design should document one global hierarchy, particularly when functions acquire locks internally.

A second deadlock-like failure: sleeping under the queue mutex

```
/* WRONG: producer can never acquire mutex to add a job. */
pthread_mutex_lock(&mutex);
while (count == 0)
    sleep(1);
pthread_mutex_unlock(&mutex);
```

This is why `pthread_cond_wait()` exists. It releases the mutex while waiting, unlike `sleep()`. You can also deadlock by waiting for a thread to join while holding a lock that the target thread needs.

07. Graceful shutdown is a state machine

Suppose the service is shutting down but has jobs already accepted. We want to stop **new submissions**, then let consumers finish the pending jobs, wake sleepers, and join all threads before destroying synchronization objects.

closed	count	Consumer action
false	0	Wait on not_empty
false	> 0	Pop a job
true	> 0	Pop existing jobs; drain queue
true	0	Exit consumer loop

The central consumer decision

```
while (q->count == 0 && !q->closed)
    pthread_cond_wait(&q->not_empty, &q->mutex);

if (q->count == 0 && q->closed) {
    pthread_mutex_unlock(&q->mutex);
    return false;  /* Finished; no future jobs. */
}
```

Close means wake everyone who could remain blocked

```
pthread_mutex_lock(&q->mutex);
q->closed = true;
pthread_cond_broadcast(&q->not_empty);
pthread_cond_broadcast(&q->not_full);
pthread_mutex_unlock(&q->mutex);
```

We broadcast to `not_empty` so idle consumers can exit; to `not_full` so blocked producers can discover closure and stop rather than wait forever. Both predicates and the closed flag are protected by the same mutex.

Normal shutdown sequence

1. Stop scheduling future producer work. **2.** Join producers once they finish submitting. **3.** Close the queue. **4.** Let consumers drain accepted jobs and exit. **5.** Join consumers. **6.** Destroy condition variables and mutex. Closing while a producer is still active is allowed in our API but may reject jobs that have not yet been accepted.

08. Thread errors, ownership, and cleanup

pthread functions return an error number

```
int rc = pthread_create(&thread, NULL, worker, &args);
if (rc != 0) {
    fprintf(stderr, "pthread_create: %s\n", strerror(rc));
    /* Clean up threads that were successfully created. */
}
```

Unlike many system calls, pthread functions generally return an error number directly rather than returning -1 and setting `errno`. Always track how many threads actually started before joining them. [1,2]

Safe ownership checklist

- Argument objects must outlive any worker that reads them; do not return pointers to expired stack objects.
- A joinable thread should be joined once (or intentionally detached). `pthread_join()` waits for a joinable thread to finish. [2]
- On partial startup failure, close the queue, wake sleepers, join threads that started, then destroy shared objects.
- Only destroy a mutex or condition variable after no thread can use it or wait on it.
- Use cooperative shutdown for queue workers. Cancellation is a separate mechanism that requires careful cleanup design; it is not our default strategy.

OS signals are not condition-variable signals

`SIGINT` / `SIGTERM` are process signals. `pthread_cond_signal()` is an in-process synchronization operation. Do not call arbitrary pthread or stdio functions from an asynchronous signal handler; a dedicated signal-waiting thread is one common safer design. [6]

09. Debugging and proving behavior

Compiler warnings and debug symbols

```
gcc -std=c11 -O0 -g -Wall -Wextra -Wpedantic -pthread 06_bounded_queue.c -o bounded_queue
./bounded_queue
```

ThreadSanitizer: find data races

```
gcc -std=c11 -O1 -g -Wall -Wextra -pthread -fsanitize=thread 02_race_bad.c -o race_bad_tsan
./race_bad_tsan
```

The intentionally incorrect example may trigger a data-race report identifying the conflicting accesses. ThreadSanitizer requires supported toolchain/runtime conditions; a runtime mapping error is not a verdict on code correctness. **Do not combine** `-fsanitize=thread` and `-fsanitize=address` in one binary. [5]

GDB: investigate a process that hangs

```
gdb --args ./bounded_queue
(gdb) run
(gdb) info threads
(gdb) thread apply all bt
(gdb) thread 2
(gdb) bt
```

A blocked stack trace can show a worker waiting inside `pthread_cond_wait()` or waiting for a mutex. Inspect which protected predicate is false and which thread could change it. GDB output depends on build symbols, libc and optimization.

Repeatability and stress testing

```
for i in $(seq 1 100); do
    timeout 5 ./bounded_queue >/dev/null || exit 1
done
echo "100 runs completed"
```

This detects some intermittent failures but cannot prove a concurrent program race-free. Review invariants, thread lifetimes, and error paths in addition to tests.

What the correct queue should demonstrate

- Output order of consumer messages may vary; that is expected.
- Once all workers have joined, `pushed == popped` and `count == 0` in the normal completed run.
- No worker remains blocked on a condition variable after the queue closes.
- No sanitizer-reported data race in a supported TSan environment (a useful check, not a formal proof).

10. One-page pthread API cheat sheet

API	Signature / essential parameters	Use
Create	`pthread_create(&tid, NULL, fn, arg)`	Start a thread at fn(arg)
Join	`pthread_join(tid, NULL)`	Wait for a joinable thread
Mutex lock	`pthread_mutex_lock(&mu)`	Acquire exclusive access
Mutex unlock	`pthread_mutex_unlock(&mu)`	Release exclusive access
Wait	`pthread_cond_wait(&cv, &mu)`	Release mutex, block, reacquire
Signal	`pthread_cond_signal(&cv)`	Wake at least one waiter if any
Broadcast	`pthread_cond_broadcast(&cv)`	Wake all current waiters
Destroy	`pthread_mutex_destroy(&mu)`	Release mutex resources after use
Destroy	`pthread_cond_destroy(&cv)`	Release condition variable after use

Common mistake to correct immediately

Mistake	Correct mental model
Signal a mutex	Signal a condition variable, e.g., not_empty
Treat signal like a stored token	The shared predicate is the truth; notifications wake rechecks
Use if around cond_wait	Use while to handle rechecks and spurious wakeups
Sleep while holding mutex	Use cond_wait to release the mutex while waiting
Close queue and exit immediately	Drain accepted work while count > 0
Protect only the writer	All conflicting accesses must obey synchronization rules
Assume equal runtime order	Only specified synchronization orders actions

Before you declare a concurrent program correct

Identify the shared objects, each object's mutex or atomic rule, every predicate, each wait/wake path, lock order, and all shutdown and error paths. Then compile with warnings, run tests repeatedly, and use a race detector when supported.

Appendix A. Complete runnable C implementations

The following listings are the **actual source files** used for build and execution checks. Their output may interleave differently on your Linux machine. Each is independently compilable with the command in the Reading Map. The deliberately broken race demonstration is labeled as such.

All examples are also provided as a separate code ZIP so you can build, edit and debug them in WSL2 rather than copying from the PDF.

A1. 01_threads.c

Thread creation, per-thread argument storage, joins and partial-startup handling.

Listing 01_threads.c, lines 1-29

```
 1  #include <pthread.h>
 2  #include <stdio.h>
 3  #include <stdlib.h>
 4  #include <string.h>
 5
 6  #define WORKERS 3
 7
 8  typedef struct {
 9      int id;
10      int jobs;
11  } WorkerArgs;
12
13  static void *worker(void *arg)
14  {
15      const WorkerArgs *a = arg;
16      for (int i = 0; i < a->jobs; ++i)
17          printf("worker %d: job %d\n", a->id, i);
18      return NULL;
19  }
20
21  int main(void)
22  {
23      pthread_t threads[WORKERS];
24      WorkerArgs args[WORKERS];
25      int started = 0;
26
27      for (int i = 0; i < WORKERS; ++i) {
28          args[i] = (WorkerArgs){ .id = i, .jobs = 3 };
29          int rc = pthread_create(&threads[i], NULL, worker, &args[i]);
```

Listing 01_threads.c, lines 30-45

```
30          if (rc != 0) {
31              fprintf(stderr, "pthread_create: %s\n", strerror(rc));
32              break;
33          }
34          started++;
35      }
36
37      for (int i = 0; i < started; ++i) {
38          int rc = pthread_join(threads[i], NULL);
39          if (rc != 0) {
40              fprintf(stderr, "pthread_join: %s\n", strerror(rc));
41              return EXIT_FAILURE;
42          }
43      }
44      return started == WORKERS ? EXIT_SUCCESS : EXIT_FAILURE;
45  }
```

A2. 02_race_bad.c

Deliberately unsafe counter: use only to study data-race diagnostics.

Listing 02_race_bad.c, lines 1-26

```c
/* INTENTIONALLY INCORRECT. Run with ThreadSanitizer. */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define LOOPS 100000
static int counter = 0;

static void *worker(void *ignored)
{
    (void)ignored;
    for (int i = 0; i < LOOPS; ++i)
        counter++;  /* Data race: unsynchronized read-modify-write */
    return NULL;
}

int main(void)
{
    pthread_t a, b;
    if (pthread_create(&a, NULL, worker, NULL) != 0) return EXIT_FAILURE;
    if (pthread_create(&b, NULL, worker, NULL) != 0) return EXIT_FAILURE;
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter=%d (expected %d, but program has UB)\n", counter, 2 * LOOPS);
    return 0;
}
```

A3. 03_mutex_counter.c

Safe counter protected by a mutex.

Listing 03_mutex_counter.c, lines 1-33

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define LOOPS 100000
static int counter = 0;
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *ignored)
{
    (void)ignored;
    for (int i = 0; i < LOOPS; ++i) {
        pthread_mutex_lock(&mutex);
        counter++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main(void)
{
    pthread_t a, b;
    if (pthread_create(&a, NULL, worker, NULL) != 0) return EXIT_FAILURE;
    if (pthread_create(&b, NULL, worker, NULL) != 0) {
        pthread_join(a, NULL);
        return EXIT_FAILURE;
    }
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter=%d (expected %d)\n", counter, 2 * LOOPS);
    pthread_mutex_destroy(&mutex);
    return counter == 2 * LOOPS ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A4. 04_condition_variable.c

Correct predicate-based producer-to-consumer notification.

Listing 04_condition_variable.c, lines 1-38

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t ready_cv = PTHREAD_COND_INITIALIZER;
static int ready = 0;
static int value = 0;

static void *consumer(void *ignored)
{
    (void)ignored;
    pthread_mutex_lock(&mutex);
    while (!ready)                    /* Recheck protected predicate. */
        pthread_cond_wait(&ready_cv, &mutex);
    int local_copy = value;           /* Protected by the same mutex. */
    pthread_mutex_unlock(&mutex);
    printf("consumed %d\n", local_copy);
    return NULL;
}

int main(void)
{
    pthread_t thread;
    if (pthread_create(&thread, NULL, consumer, NULL) != 0)
        return EXIT_FAILURE;

    pthread_mutex_lock(&mutex);
    value = 42;
    ready = 1;                       /* Update state BEFORE notifying. */
    pthread_cond_signal(&ready_cv);
    pthread_mutex_unlock(&mutex);

    pthread_join(thread, NULL);
    pthread_cond_destroy(&ready_cv);
    pthread_mutex_destroy(&mutex);
    return EXIT_SUCCESS;
}
```

A5. 05_lock_order.c

Two workers that acquire locks A then B, avoiding cyclic acquisition.

Listing 05_lock_order.c, lines 1-34

```
 1  #include <pthread.h>
 2  #include <stdio.h>
 3  #include <stdlib.h>
 4
 5  static pthread_mutex_t a = PTHREAD_MUTEX_INITIALIZER;
 6  static pthread_mutex_t b = PTHREAD_MUTEX_INITIALIZER;
 7
 8  static void *worker(void *arg)
 9  {
10      int id = *(int *)arg;
11      pthread_mutex_lock(&a);          /* Every thread: A before B. */
12      pthread_mutex_lock(&b);
13      printf("worker %d holds A and B\n", id);
14      pthread_mutex_unlock(&b);        /* Reverse release is conventional. */
15      pthread_mutex_unlock(&a);
16      return NULL;
17  }
18
19  int main(void)
20  {
21      int ids[2] = { 1, 2 };
22      pthread_t threads[2];
23      if (pthread_create(&threads[0], NULL, worker, &ids[0]) != 0)
24          return EXIT_FAILURE;
25      if (pthread_create(&threads[1], NULL, worker, &ids[1]) != 0) {
26          pthread_join(threads[0], NULL);
27          return EXIT_FAILURE;
28      }
29      pthread_join(threads[0], NULL);
30      pthread_join(threads[1], NULL);
31      pthread_mutex_destroy(&b);
32      pthread_mutex_destroy(&a);
33      return EXIT_SUCCESS;
34  }
```

A6. 06_bounded_queue.c

Complete bounded queue: multiple producers, consumers, shutdown and cleanup.

Listing 06_bounded_queue.c, lines 1-42

```c
/* Complete multi-producer, multi-consumer queue with graceful shutdown. */
#include <pthread.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define CAPACITY 3
#define PRODUCERS 2
#define CONSUMERS 3
#define JOBS_PER_PRODUCER 20

typedef struct {
    int items[CAPACITY];
    size_t head, tail, count;
    bool closed;
    size_t pushed, popped;
    pthread_mutex_t mutex;
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
} Queue;

static Queue queue = {
    .mutex = PTHREAD_MUTEX_INITIALIZER,
    .not_empty = PTHREAD_COND_INITIALIZER,
    .not_full = PTHREAD_COND_INITIALIZER
};

/* Return false if the queue was closed before this job was accepted. */
static bool queue_push(Queue *q, int job)
{
    pthread_mutex_lock(&q->mutex);
    while (q->count == CAPACITY && !q->closed)
        pthread_cond_wait(&q->not_full, &q->mutex);

    if (q->closed) {
        pthread_mutex_unlock(&q->mutex);
        return false;
    }
    q->items[q->tail] = job;
    q->tail = (q->tail + 1) % CAPACITY;
    q->count++;
```

Listing 06_bounded_queue.c, lines 43-84

```
43      q->pushed++;
44      pthread_cond_signal(&q->not_empty);
45      pthread_mutex_unlock(&q->mutex);
46      return true;
47  }
48
49  /* Return false only when closed AND empty; otherwise give one job. */
50  static bool queue_pop(Queue *q, int *job)
51  {
52      pthread_mutex_lock(&q->mutex);
53      while (q->count == 0 && !q->closed)
54          pthread_cond_wait(&q->not_empty, &q->mutex);
55
56      if (q->count == 0 && q->closed) {
57          pthread_mutex_unlock(&q->mutex);
58          return false;
59      }
60      *job = q->items[q->head];
61      q->head = (q->head + 1) % CAPACITY;
62      q->count--;
63      q->popped++;
64      pthread_cond_signal(&q->not_full);
65      pthread_mutex_unlock(&q->mutex);
66      return true;
67  }
68
69  static void queue_close(Queue *q)
70  {
71      pthread_mutex_lock(&q->mutex);
72      q->closed = true;                     /* No new pushes. */
73      pthread_cond_broadcast(&q->not_empty); /* Wake all consumers. */
74      pthread_cond_broadcast(&q->not_full);  /* Wake blocked producers. */
75      pthread_mutex_unlock(&q->mutex);
76  }
77
78  static void *producer(void *arg)
79  {
80      int id = *(int *)arg;
81      for (int i = 0; i < JOBS_PER_PRODUCER; ++i) {
82          if (!queue_push(&queue, id * 1000 + i))
83              break;                        /* Closed: stop submitting. */
84      }
```

Listing 06_bounded_queue.c, lines 85-126

```
 85      return NULL;
 86  }
 87
 88  static void *consumer(void *arg)
 89  {
 90      int id = *(int *)arg;
 91      int job;
 92      while (queue_pop(&queue, &job)) {
 93          /* Real application: process job outside the queue mutex. */
 94          printf("consumer %d processed %d\n", id, job);
 95      }
 96      return NULL;
 97  }
 98
 99  static int join_all(pthread_t *threads, int n)
100  {
101      int ok = 1;
102      for (int i = 0; i < n; ++i) {
103          int rc = pthread_join(threads[i], NULL);
104          if (rc != 0) {
105              fprintf(stderr, "pthread_join: %s\n", strerror(rc));
106              ok = 0;
107          }
108      }
109      return ok;
110  }
111
112  int main(void)
113  {
114      pthread_t producers[PRODUCERS], consumers[CONSUMERS];
115      int producer_ids[PRODUCERS], consumer_ids[CONSUMERS];
116      int np = 0, nc = 0;
117      int rc, ok = 1;
118
119      /* Start consumers first; each can wait for a future job. */
120      for (int i = 0; i < CONSUMERS; ++i) {
121          consumer_ids[i] = i;
122          rc = pthread_create(&consumers[i], NULL, consumer,
123                              &consumer_ids[i]);
124          if (rc != 0) {
125              fprintf(stderr, "create consumer: %s\n", strerror(rc));
126              ok = 0;
```

Listing 06_bounded_queue.c, lines 127-164

```
127             goto shutdown;
128         }
129         nc++;
130     }
131     for (int i = 0; i < PRODUCERS; ++i) {
132         producer_ids[i] = i + 1;
133         rc = pthread_create(&producers[i], NULL, producer,
134                             &producer_ids[i]);
135         if (rc != 0) {
136             fprintf(stderr, "create producer: %s\n", strerror(rc));
137             ok = 0;
138             goto shutdown;
139         }
140         np++;
141     }
142
143     /* Normal path: producers finish; then close and drain the queue. */
144     if (!join_all(producers, np)) ok = 0;
145     np = 0;  /* Already joined, so never join these handles twice. */
146
147 shutdown:
148     queue_close(&queue);                /* Also handles startup failure. */
149     if (!join_all(producers, np)) ok = 0;
150     if (!join_all(consumers, nc)) ok = 0;
151
152     printf("summary: pushed=%zu popped=%zu left=%zu\n",
153            queue.pushed, queue.popped, queue.count);
154     if (queue.count != 0 || queue.pushed != queue.popped)
155         ok = 0;
156     /* On the normal successful path all intended jobs were submitted. */
157     if (ok && queue.pushed != PRODUCERS * JOBS_PER_PRODUCER)
158         ok = 0;
159
160     pthread_cond_destroy(&queue.not_empty);
161     pthread_cond_destroy(&queue.not_full);
162     pthread_mutex_destroy(&queue.mutex);
163     return ok ? EXIT_SUCCESS : EXIT_FAILURE;
164 }
```

A7. 07_atomic_counter.c

Optional C11 atomic counter with relaxed increments.

Listing 07_atomic_counter.c, lines 1-30

```
/* Optional extension: simple shared counters can use C11 atomics. */
#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>

#define LOOPS 100000
static atomic_int counter = ATOMIC_VAR_INIT(0);

static void *worker(void *ignored)
{
    (void)ignored;
    for (int i = 0; i < LOOPS; ++i)
        atomic_fetch_add_explicit(&counter, 1, memory_order_relaxed);
    return NULL;
}

int main(void)
{
    pthread_t a, b;
    if (pthread_create(&a, NULL, worker, NULL) != 0) return EXIT_FAILURE;
    if (pthread_create(&b, NULL, worker, NULL) != 0) {
        pthread_join(a, NULL);
        return EXIT_FAILURE;
    }
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("atomic counter=%d\n", atomic_load(&counter));
    return EXIT_SUCCESS;
}
```

References / primary documentation

The technical descriptions are based on published POSIX / Linux manual pages and GCC documentation. The linked references are the authoritative source for API signatures and precise guarantees; platform-specific details should be checked against the documentation installed on your system.

[1] pthread_create(3), Linux man-pages
https://man7.org/linux/man-pages/man3/pthread_create.3.html

[2] pthread_join(3), Linux man-pages
https://man7.org/linux/man-pages/man3/pthread_join.3.html

[3] pthread_cond_wait(3p), POSIX Programmer's Manual
https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html

[4] pthread_cond_signal / pthread_cond_broadcast(3p), POSIX
https://man7.org/linux/man-pages/man3/pthread_cond_broadcast.3p.html

[5] GCC instrumentation and ThreadSanitizer
https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html

[6] signal-safety(7), Linux man-pages
https://man7.org/linux/man-pages/man7/signal-safety.7.html

[7] pthread_mutex_lock(3p), POSIX Programmer's Manual
https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html

Run locally - relevant man pages

```
man 3 pthread_create
man 3 pthread_join
man 3 pthread_cond_wait
man 3 pthread_mutex_lock
```

Course sequence

Now: Read this reference and run the code as needed. **Next message:** a strict assessment of the Module 4 concepts and code tracing. **Following message:** a self-directed integration project specification (requirements, interfaces, failure cases, and QA).